Capture filters vs display filters in Wireshark

Wireshark is a network protocol analyzer used to capture, inspect, and investigate network packets.

A SOC analyst uses Wireshark to identify suspicious communication, investigate incidents, and understand network behaviour.

1. Filters

There are two important types of filters.

1.1. Capture filters

- Capture filters determine which packets Wireshark collects.
- They are applied before or during packet capture.
- They use Berkeley Packet Filter (BPF) syntax.
- For example, if you only want to capture traffic involving a particular IP address, you can apply a capture filter before starting the capture.

Example: host 192.168.1.10
This captures packets to or from that IP address.

Key point: Packets excluded by a capture filter are not saved in that capture.

1.2. Display filters

- Display filters determine which packets are displayed from an existing capture.
- They are applied after packets have been captured.
- They use Wireshark's own filter syntax.

Example: ip.addr == 192.168.1.10
This displays packets involving the specified IP address.

Key point: Display filters hide packets from view but do not delete them from the capture file.

1.3. Capture filters vs display filters

Feature	Capture filter	Display filter
When applied	Before/during capture	After capture
Purpose	Controls which packets are collected	Controls which packets are displayed
Syntax	BPF	Wireshark filter syntax

Can reduce capture file size?	Yes	No
Can recover excluded packets?	No	Not applicable; captured packets remain available
Example	host 192.168.1.10	ip.addr == 192.168.1.10

1.4. Ten working Wireshark filter examples

Here are ten practical examples. The capture and display filters are separate syntaxes, so do not interchange them.

#	**Objective**	**Capture filter**	**Display filter**
1	Traffic involving an IP	host 192.168.1.10	ip.addr == 192.168.1.10
2	Traffic from an IP	src host 192.168.1.10	ip.src == 192.168.1.10
3	Traffic to an IP	dst host 192.168.1.20	ip.dst == 192.168.1.20
4	HTTP traffic	tcp port 80	http
5	HTTPS traffic	tcp port 443	tcp.port == 443
6	DNS traffic	udp port 53	dns
7	ICMP traffic	icmp	icmp
8	SSH traffic	tcp port 22	tcp.port == 22
9	SMB traffic	tcp port 445	tcp.port == 445
10	Traffic between two hosts	host 192.168.1.10 and host 192.168.1.20	ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20

Note: The DNS capture filter above captures conventional UDP DNS, not all DNS. TCP DNS and DNS over HTTPS require additional consideration.

1.5. Five additional useful display filters

These are particularly useful during SOC investigations.

i. Find TCP SYN packets without ACK
tcp.flags.syn == 1 && tcp.flags.ack == 0

Use case: Identify initial TCP connection attempts and possible SYN scanning.

ii. Find TCP reset packets
tcp.flags.reset == 1

Use case: Investigate rejected or abruptly terminated connections.

iii. Find failed HTTP responses
http.response.code >= 400

Use case: Investigate HTTP client and server errors.

iv. Find DNS queries
dns.flags.response == 0

Use case: Identify domains being requested by clients.

v. Find retransmissions
tcp.analysis.retransmission

Use case: Investigate packet loss, congestion, or unusual retransmission behaviour.

Practical SOC investigation example

Suppose your SIEM reports that an employee's workstation is communicating with a suspicious external IP address.

You receive a PCAP file.

Your investigation could look like this:

i. Filter the suspicious IP:
ip.addr == 203.0.113.50

ii. Identify DNS queries:
dns

iii. Identify HTTP requests:
http.request

iv. Identify TLS handshakes:
tls.handshake

v. Identify unusual TCP connection attempts:
tcp.flags.syn == 1 && tcp.flags.ack == 0

vi. Inspect packet details and determine which internal system initiated the connection.

This helps establish whether the traffic is legitimate, suspicious, or potentially malicious.